Storage Allocation and Memory Techniques



Presented by : **Ayush Kumar, 24190503015**

Rahul Kumar, 24190503042

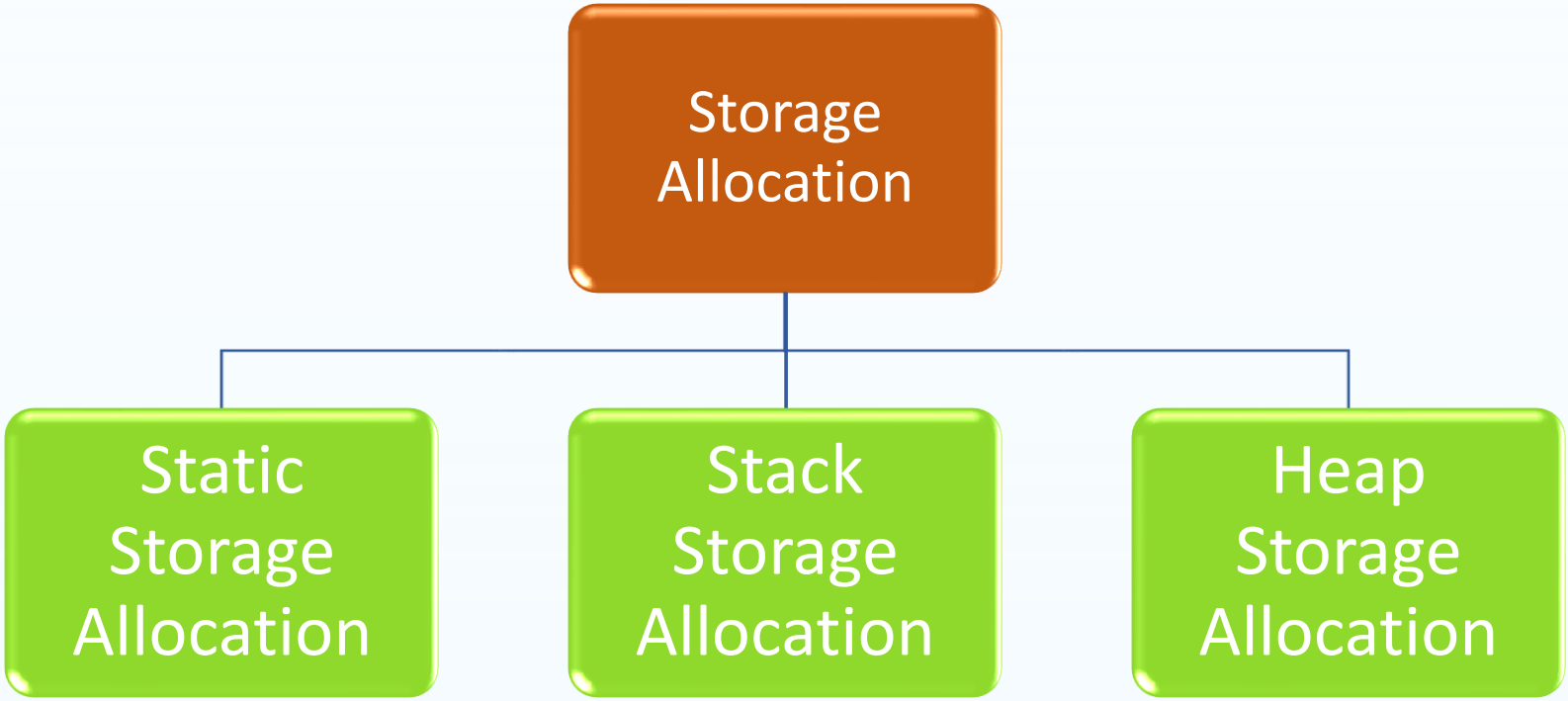
Deepak Kumar, 24190503057

01

Storage Allocation Techniques

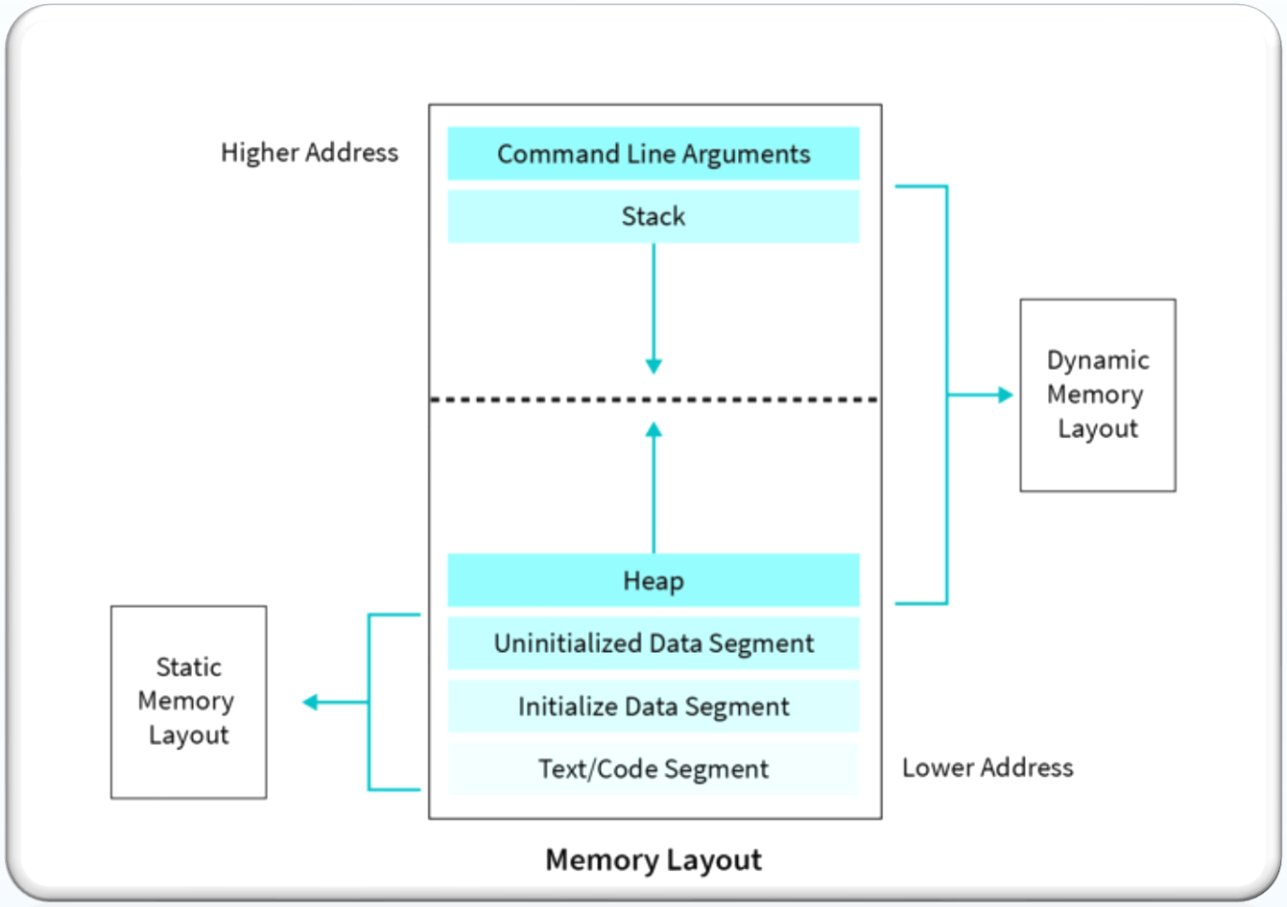
Storage Allocation Techniques

Storage Allocation Technique is of 3 Types :-



Static Storage Allocation

- In Static Allocation, Programmes or Processes are bounded to a storage location.
- The Memory is created at compile time then the memory will be created in static area and only once.
- Static Allocation supports the dynamic data structure that means memory is created only at the compile time and deallocate after the programme completion.



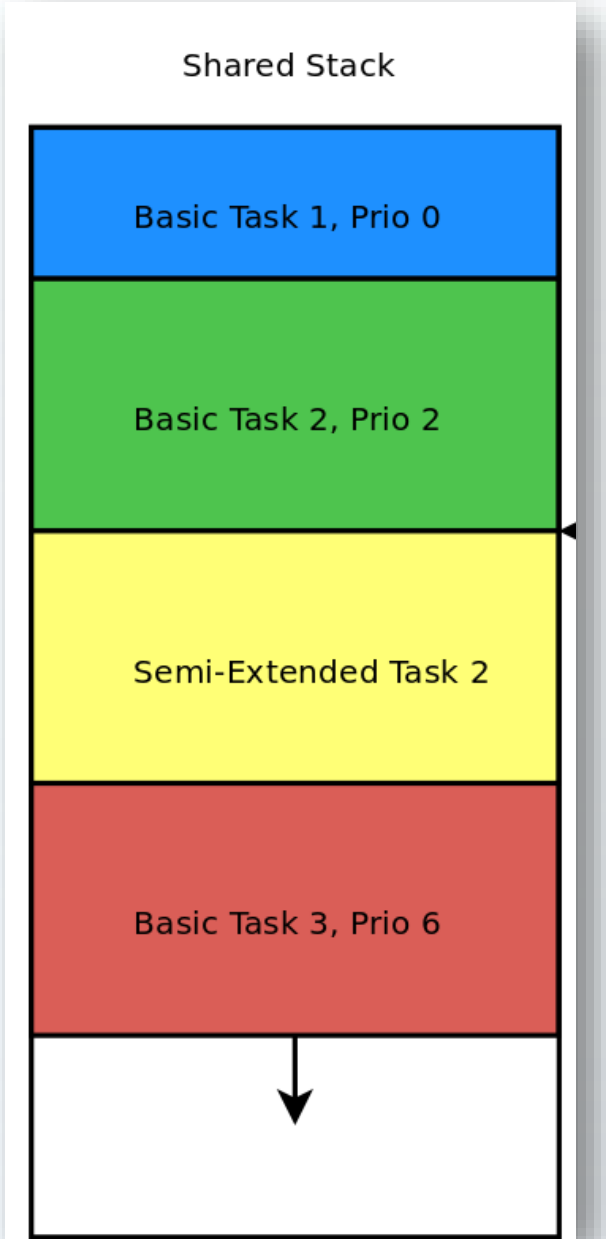
✅ Advantages

1. **Faster Execution:**
   - Since memory is allocated at compile time, there's no overhead of allocation/deallocation during runtime.
2. **Predictability**:
   - The memory layout is fixed and known, which can make debugging and optimization easier.
3. **No Fragmentation:**
   - As memory is allocated once and doesn't change during execution, fragmentation is not an issue.
4. **Simple Implementation:**
   - Static memory allocation is simpler to implement compared to dynamic allocation.

❌ Disadvantages

1. **Wasted Memory :**
   - If the allocated memory is more than needed, the extra memory remains unused.
2. **Lack of Flexibility:**
   - Memory size is fixed at compile time, so you can't adjust it during runtime based on program needs.
3. **Limited Use in Complex Programs:**
   - Not suitable for programs requiring dynamic data structures like linked lists, trees, or resizing arrays.
4. **Stack Overflow Risk:**
   - In some systems, large static allocations may exceed stack size, leading to overflow.

Stack Storage Allocation



- In Stack Allocation , the storage is organised as stack.
- Activation record of Program or Processes are pushed and popped into the stack.
- It works on the basis of LIFO and this Allocation supports the recursive process.
- Each and Every Time Fresh Memory is Allocated.

✅ Advantages

1. **Fast Allocation and Deallocation:**
   - Memory is allocated and freed automatically when functions are called and return.
2. **Efficient Use of Memory:**
   - Stack allocation is done in a Last In, First Out (LIFO) order, which is simple and efficient.
3. **No Memory Leaks:**
   - Since memory is freed automatically, there's less risk of forgetting to deallocate.
4. **Low Overhead:**
   - No need for complex memory management logic.

❌ Disadvantages

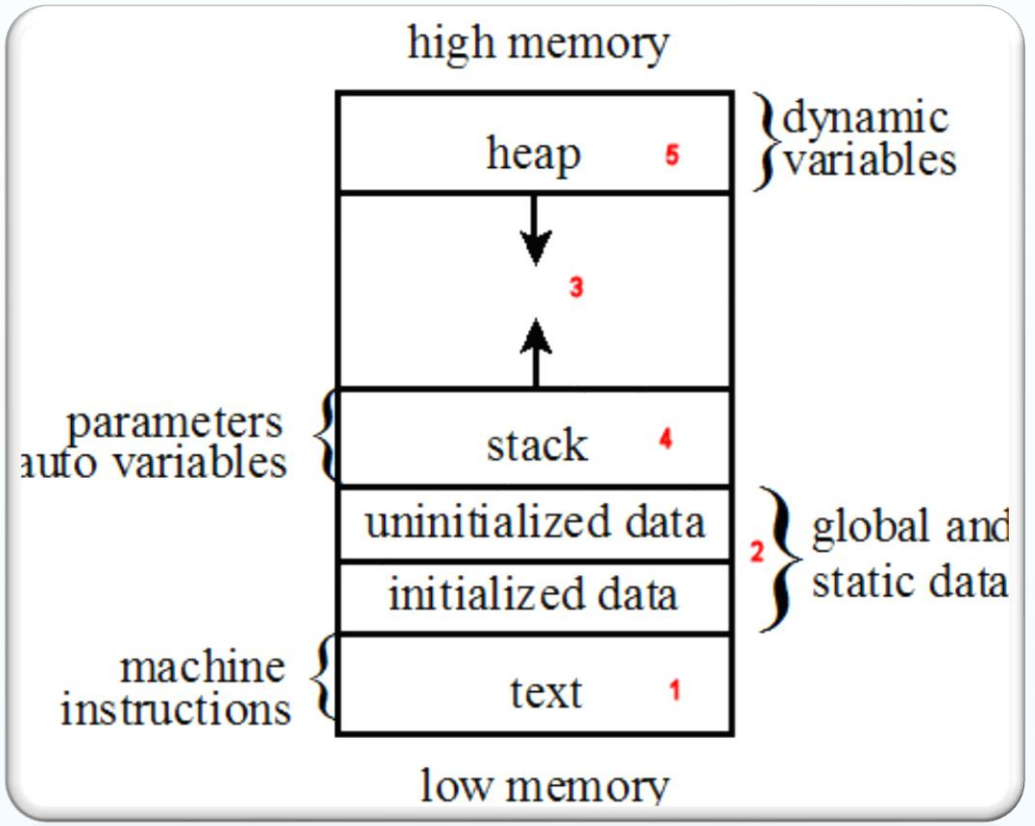
1. **Limited Size:**
   - Stack size is usually small and fixed. Allocating large data structures can cause **stack overflow**.
2. **Short Lifetime:**
   - Variables allocated on the stack only exist during the function's execution. They disappear after the function returns.
3. **Lack of Flexibility:**
   - Memory cannot be resized or persist beyond function scope.
4. **Not Suitable for Dynamic Data Structures:**
   - Structures like dynamic arrays, linked lists, or trees typically require **heap** allocation.

Heap Storage Allocation

Allocation and de allocation memory can be done at any time and at any place depending upon the user's requirement.

Heap allocation is used to allocate memory to the various processes dynamically and when the processes are no more used then we could delete it.

Heap storage allocation is the most flexible allocation scheme.



✅ Advantages

1. **Dynamic Allocation:**
   - Memory can be allocated or resized during program execution(e.g., using malloc in c, new in C++/Java).
2. **Flexible Lifetime:**
   - Data in heap memory can persist beyond the function that created it, allowing shared and long-term access.
3. **Good for Complex Structures:**
   - Suitable for dynamic data structures like linked lists, trees, graphs, etc.
4. **Large Memory Pool:**
   - Generally has more available space than the stack, making it better for large objects.

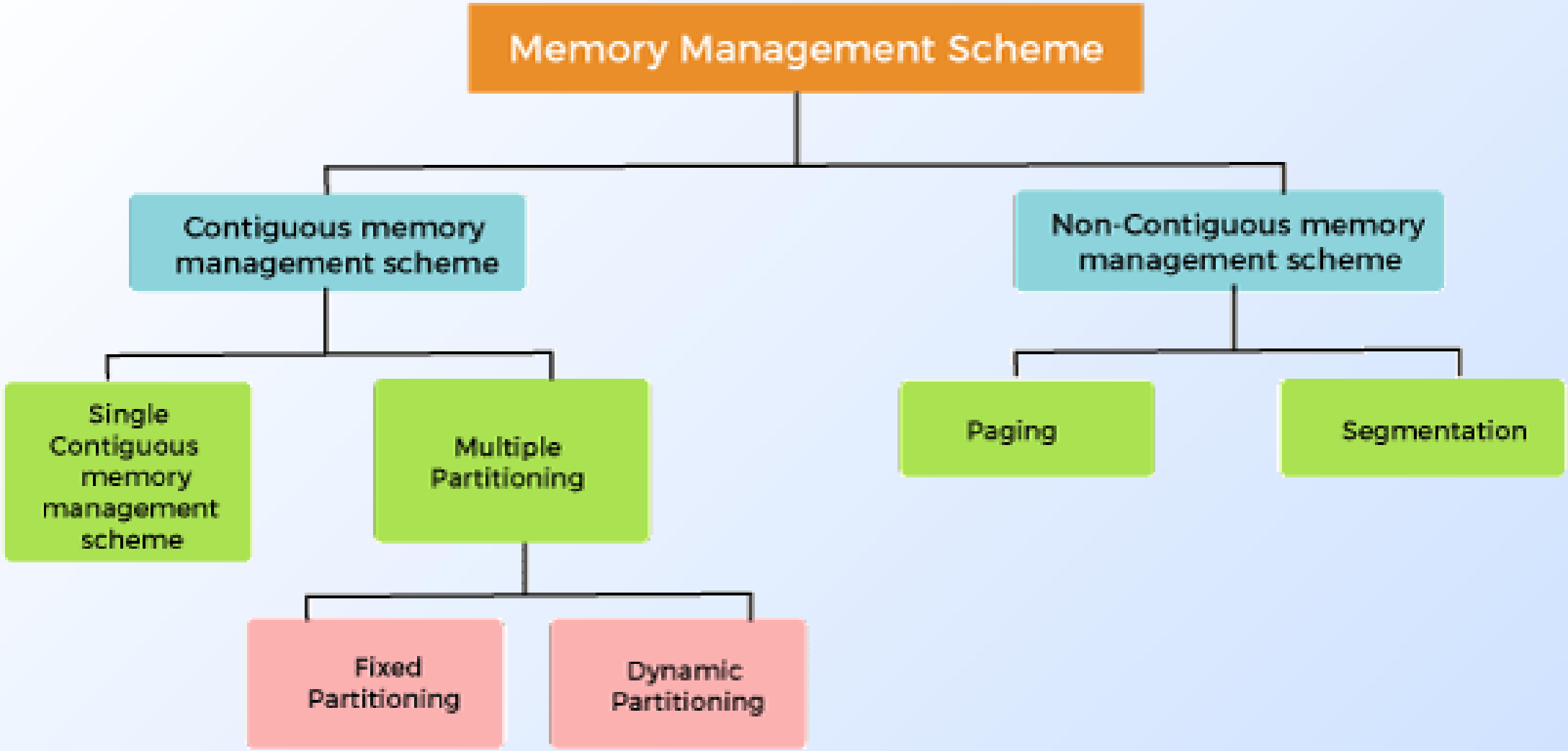
❌ Disadvantages

1. **Slower Allocation:**
   - Allocating and freeing memory on the heap is slower than on the stack due to bookkeeping overhead.
2. **Manual Memory Management (in some languages):**
   - In languages like C/C++, forgetting to free memory can cause **memory leaks**.
3. **Fragmentation:**
   - Heap memory can become fragmented over time, reducing performance and available space.
4. **Security Risk:**
   - Improper use can lead to bugs like **dangling pointers**, **buffer overflows**, and **double frees**.

02

Memory Management Techniques

Memory Management Technique :

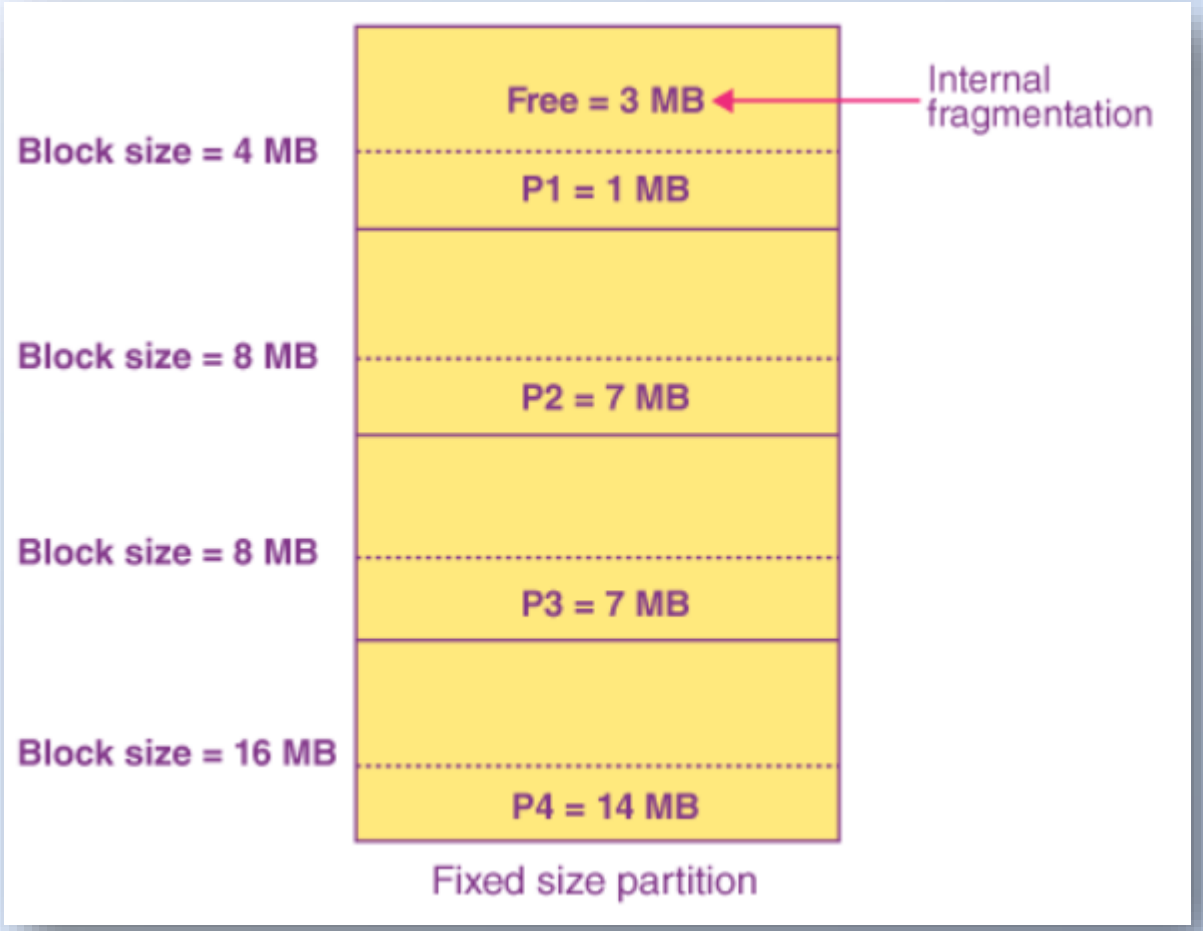


Classification of memory management schemes

Fixed Partition in Contiguous Memory Management :

Fixed Partitioning (Static Partitioning)

- **Definition**: Fixed partitioning is a memory management scheme where the main memory is divided into a **fixed number of partitions before execution**.
- **Number of Partition are Fixed.**
- **Each partition can hold only one process at a time.**
- **Size of Each partitions are predefined at system boot-up.**
- **Fixed Partition is Contiguous Allocation so Spanning is not Possible.**
- Used in **early operating systems** like IBM OS/360.



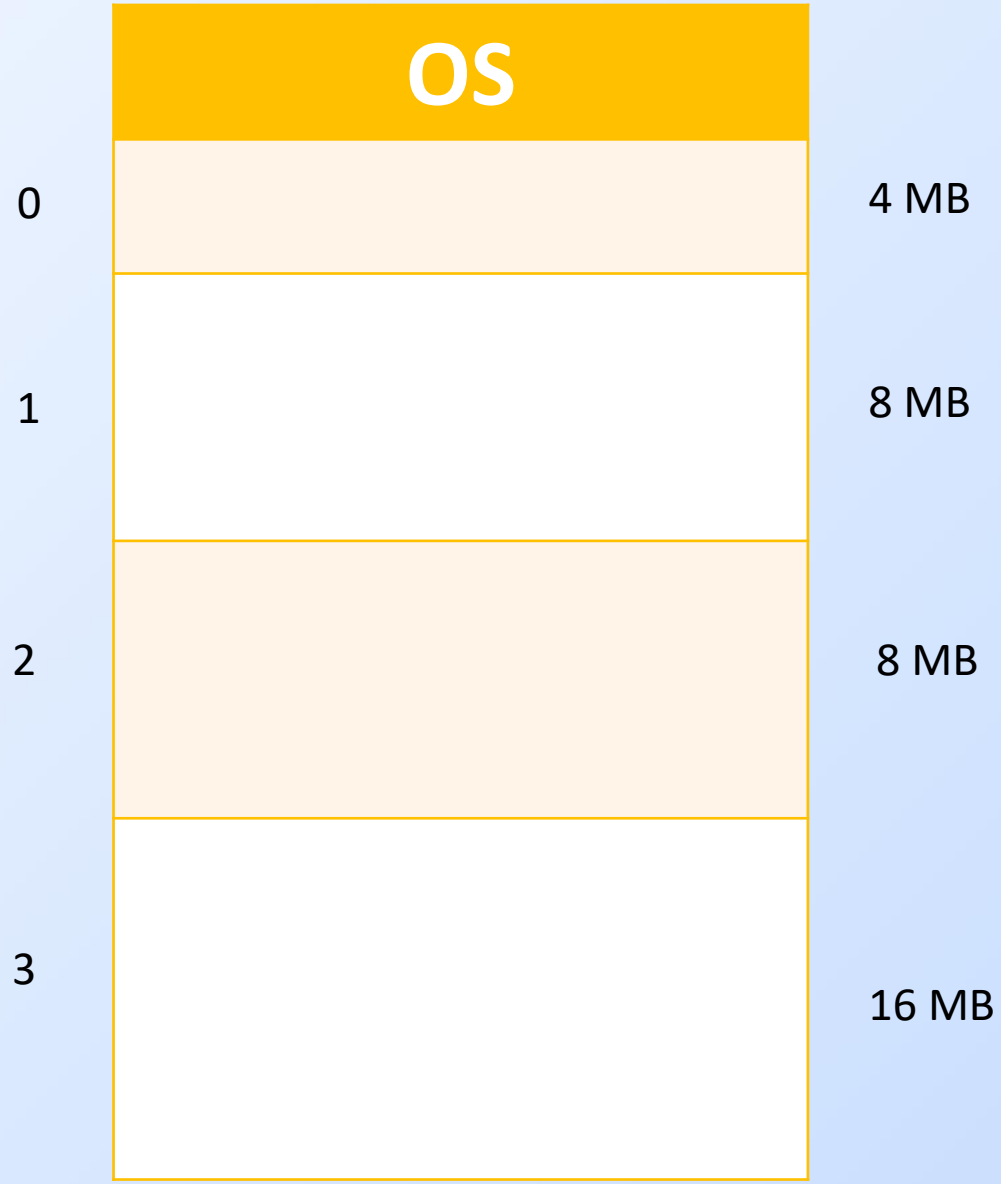
Fixed Partition in Contiguous Memory Management :

Internal Fragmentation :

Definition : Occurs when a process is allocated more memory than it actually requires, leaving unused space **inside** the partition.

External Fragmentation :

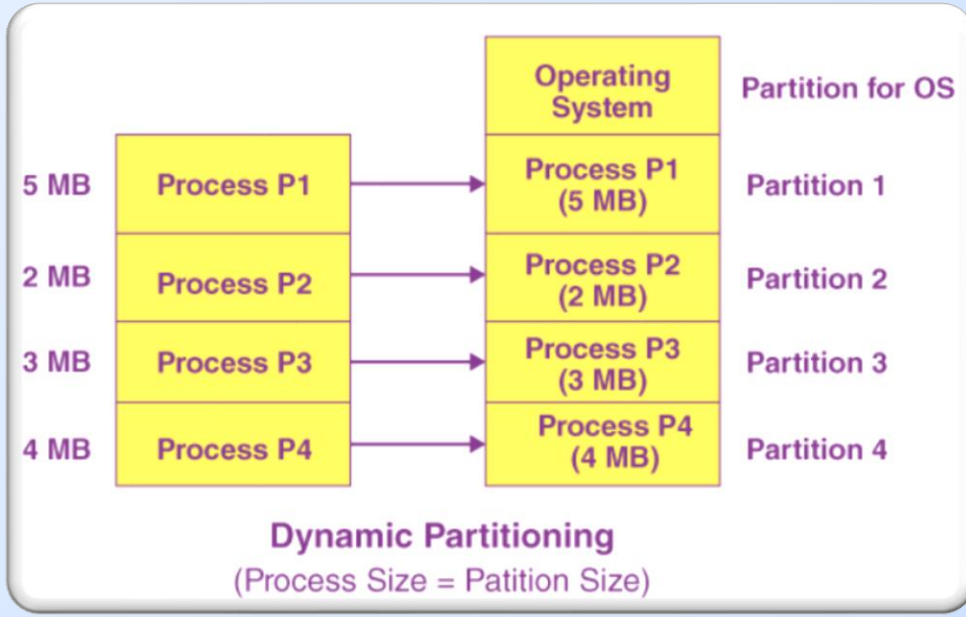
Definition : Happens when free memory is available but is scattered in **non-contiguous** small blocks, preventing large processes from being allocated.



Dynamic Partition in Contiguous Memory Management :

Dynamic Partitioning :

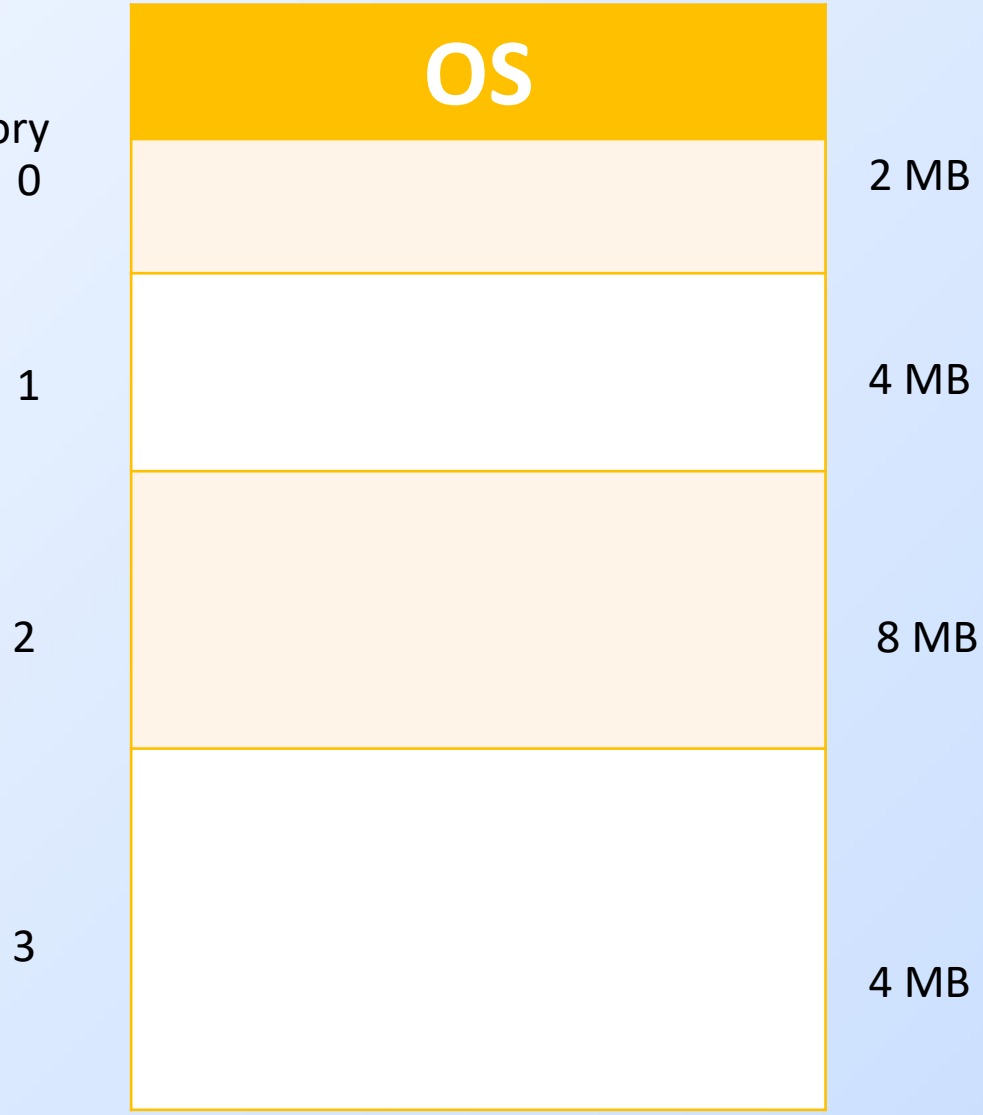
- **No fixed partition sizes**: Memory is divided dynamically based on the size of the process at runtime.
- **Memory is allocated exactly as required**: Reduces internal fragmentation significantly.
- **Partitions are created when needed**: Each process gets a separate partition tailored to its size.
- **Better memory utilization** compared to fixed partitioning.



Dynamic Partition in Contiguous Memory Management :

Dynamic partitioning (also called **variable partitioning**) is a memory allocation technique where memory is divided into variable-sized partitions based on the size of incoming processes. Unlike fixed partitioning, partition sizes are not predetermined and change dynamically.





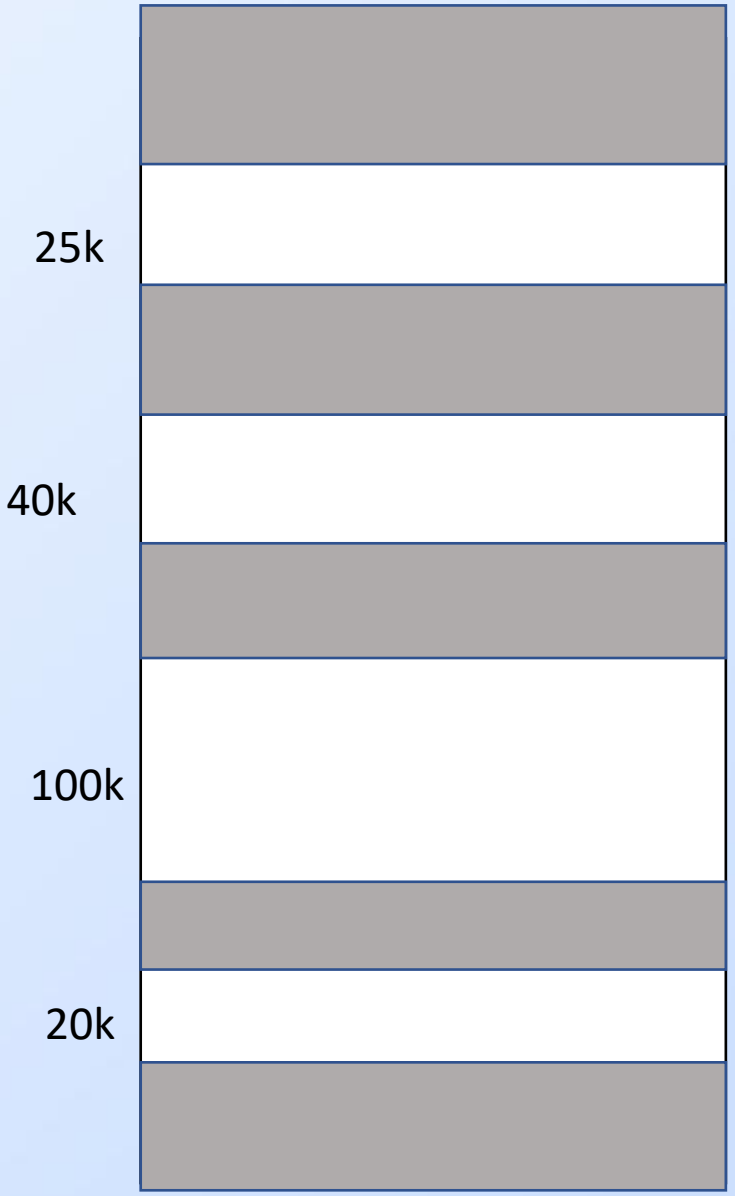
Memory Allocation Methods :

First-Fit: Allocate the first hole that is big enough.

Next-Fit: Same as first fit but Start search always from last allocated hole.

Best Fit: Allocate the smallest hole that is big enough.

Worst Fit: Allocate the largest hole.




Thank
You